



Robotics & Computer Vision

Robot Kinematics & Control

Week 6 — Bridging Vision to Robot Action: from object pose to control command

Week 6 Learning Objectives

1

What does a robot need?

Understand what comes after vision has located the object

2

Core Concepts

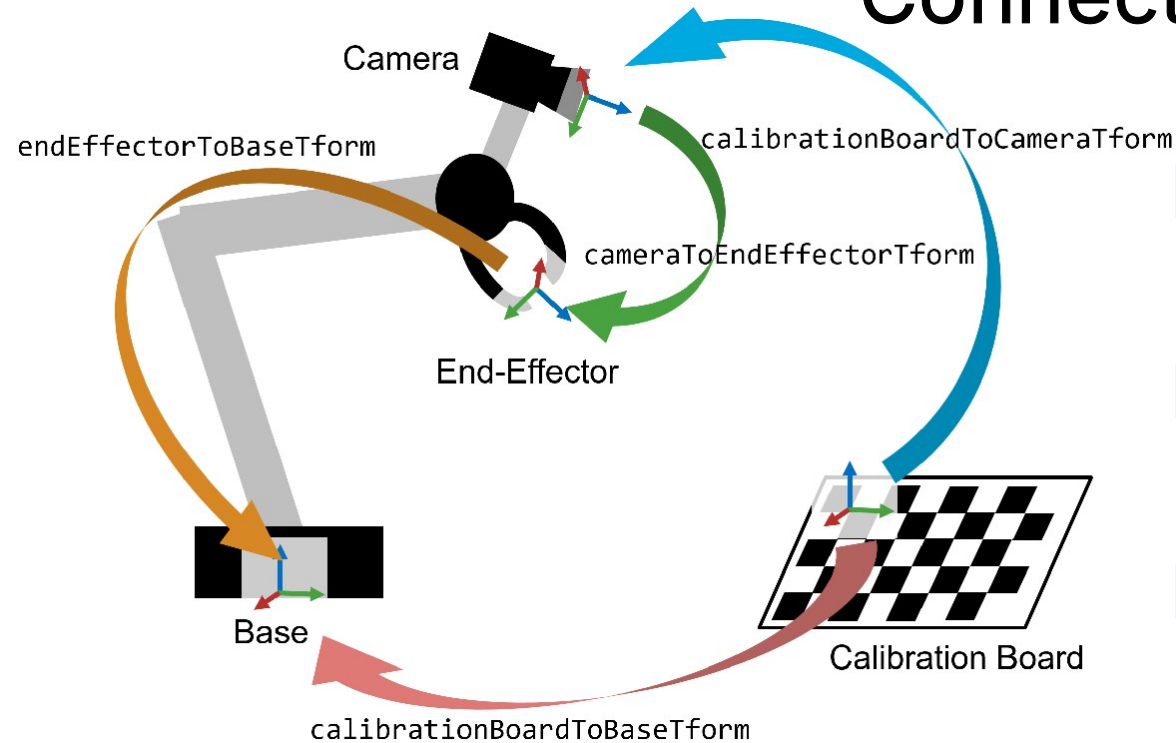
Grasp joint space, task space, FK, IK, and Jacobian

3

Build the Bridge

Connect pose in camera frame to robot motion commands

Connection to Previous Weeks



1

Week 3

Centroid & image coordinates

2

Week 4

Object detection & bounding box

3

Week 5

6D pose in camera frame

4

Week 6

Convert target to robot command

What Comes Next



Week 7

Deep dive into Forward Kinematics



Week 8

Deep dive into Inverse Kinematics



Week 9

Camera-to-robot mapping & hand-eye calibration

Why Week 6 Stays at the Overview Level

FK and IK will be studied in depth in Weeks 7 and 8. This week focuses on system intuition and integration flow.

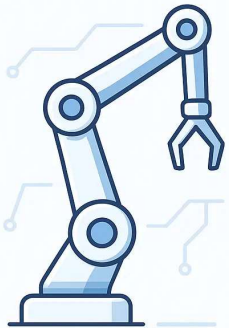
The Goal

Help students see the **end-to-end pipeline** before diving into mathematical details. Understanding the architecture first makes the math more meaningful later.

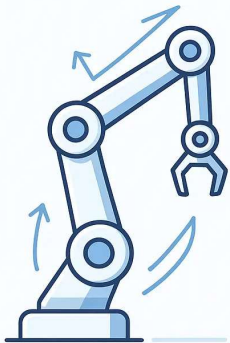
 Breadth before depth — the map before the territory.

Kinematics, Dynamics & Control

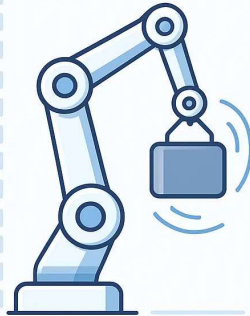
GEOMETRY



KINEMATICS



DYNAMICS



Kinematics

Geometry of motion — positions and orientations, no forces involved

Dynamics

Forces and torques — how mass and inertia affect motion

Control

Issuing commands so the robot tracks the desired target accurately

What Does "Vision $\rightarrow$ Robot" Actually Mean?

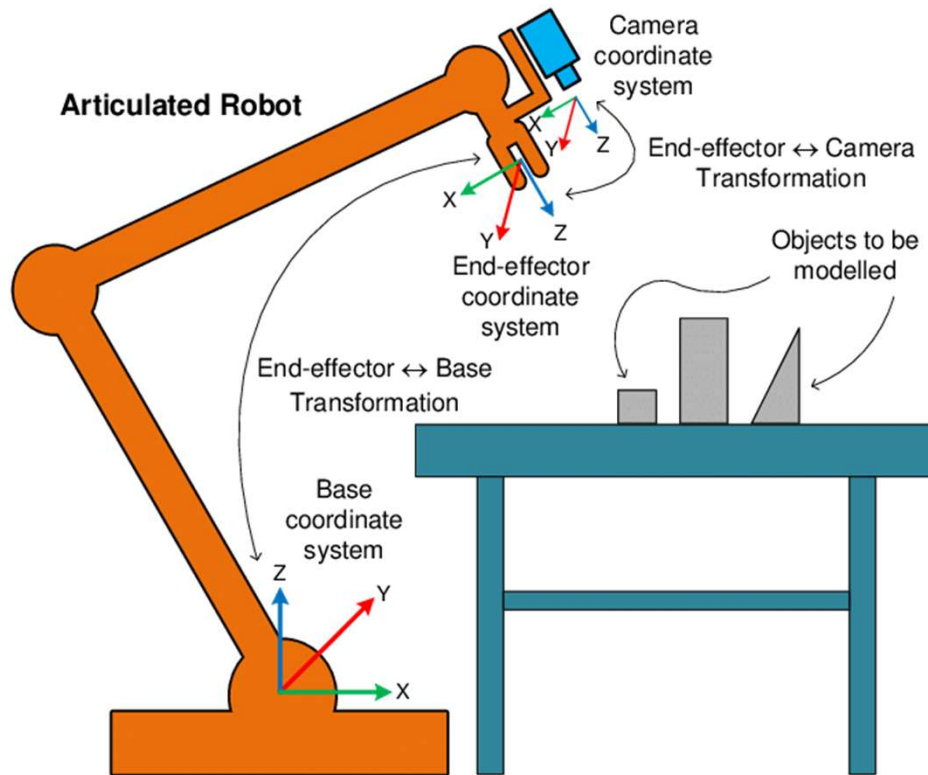
- From "where is the object?" to "how should each joint rotate?"
- From camera frame coordinates to robot base frame coordinates
- From a task-space target to a joint-space command

Part 1

Frames & Pose

Understanding coordinate systems — the language robots use to describe the world

Coordinate Frames



World Frame

Global reference for the entire environment

Robot Base Frame

Fixed to the robot's mounting point

Camera Frame

Origin at the camera's optical center

Tool (TCP) Frame

At the tip of the end-effector

Object Frame

Attached to the target object

Why Frames Matter

Same Object, Different Coordinates

The same physical object has completely different coordinate values depending on which frame you use to describe it

Vision Output = Camera Frame

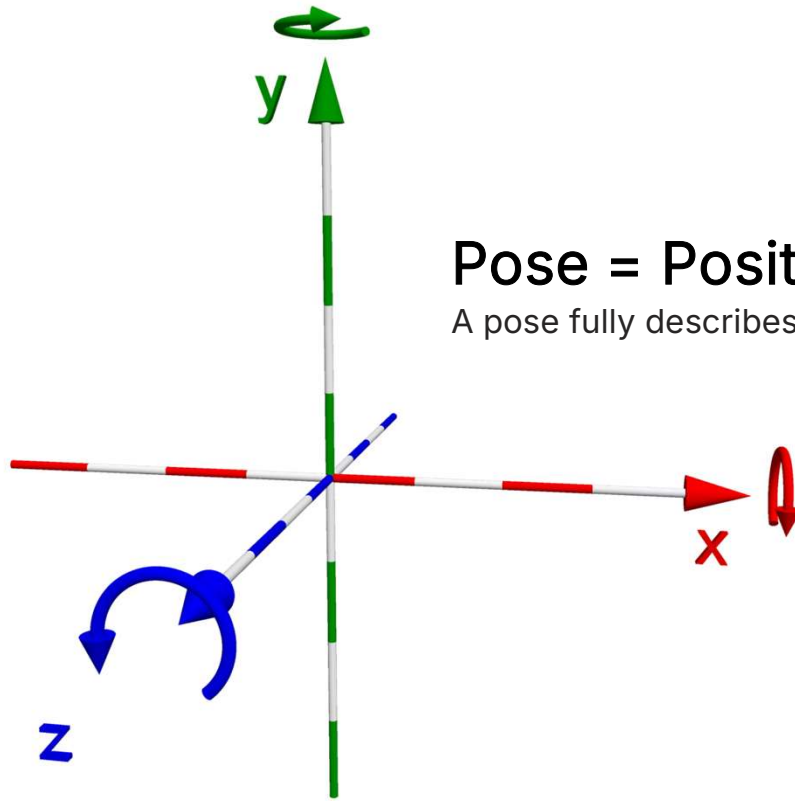
Object detection and 6D pose estimation typically output results in the camera's own coordinate frame

Robot Needs Base Frame

To reach the object, the robot requires the target expressed in its base frame — a frame transformation is mandatory

 Incorrect frame transformation = the entire pipeline fails, even if vision is perfect.

What Is a Pose?



Pose = Position + Orientation

A pose fully describes **where** an object is and **which way** it is facing in 3D space.

Rotation Matrix

Describing Orientation in 3D

A rotation matrix R encodes the orientation of one frame relative to another. It belongs to the special orthogonal group $SO(3)$.

- 3×3 matrix with orthonormal columns
- Determinant = +1 (pure rotation, no reflection)
- Preserves lengths and angles

 R is the core building block of pose. Every homogeneous transform contains R as its upper-left 3×3 block.

$$R \in SO(3), \quad R^T R = I, \quad \det(R) = 1$$

Translation Vector

Describing Position

The translation vector $\mathbf{p}$ describes the origin of a child frame expressed in the parent frame's coordinates.

$$\mathbf{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

Combined with R

Together, $\mathbf{R}$ and $\mathbf{p}$ form a complete pose description. This pair is the foundation of all frame transformations in robotics.

□ Think of $\mathbf{R}$ as the rotation dial and $\mathbf{p}$ as the position pin — you need both to fully specify a frame.

Homogeneous Transform

The 4×4 Standard

A homogeneous transformation matrix T packs rotation and translation into a single 4×4 matrix — the standard language for chaining multiple frame transformations.

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Why It Matters

- Chaining transforms = simple matrix multiplication
- Critical for camera-to-robot mapping
- Used in every robotics toolbox (e.g., Robotics Toolbox for Python)

Chaining Transforms



Base to Camera

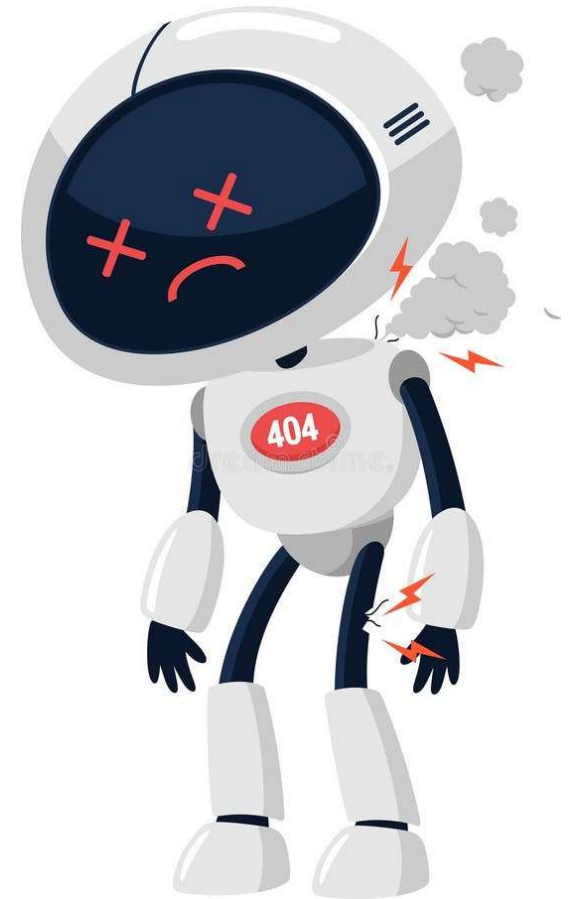
Camera to Object

Base to Object

The final pose is the product of successive transforms. A serial robot arm is naturally described this way — each link adds one transform. This chain is the geometric core idea behind Forward Kinematics.

Wrong frame = Wrong robot

Converting vision output to the robot's base frame is not optional. It is the prerequisite for every grasp and manipulation task. A correctly calibrated transform chain is the foundation of the entire pipeline.



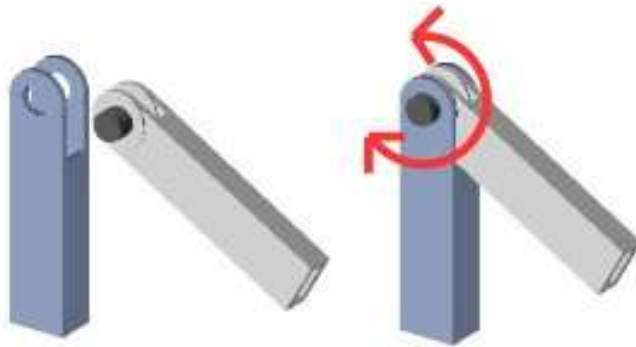
Part 2

Joint Space & Task Space

The two worlds a robot lives in — and the bridge between them

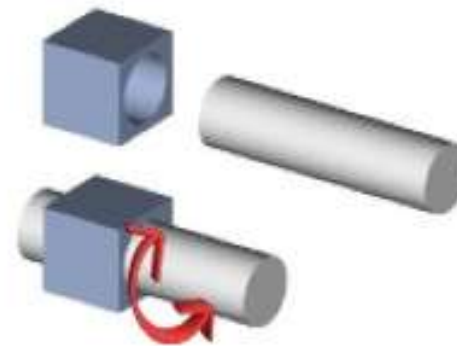
Joint Variables

All joint variables are collected into the **configuration vector $\mathbf{q}$** , which fully describes the robot's posture.



Revolute Joint

Uses angle θ — rotates around an axis



Prismatic Joint

Uses displacement d — slides along an axis

Joint Space

The Space of Joint Values

Every point in joint space represents one specific robot configuration — a unique combination of all joint angles or displacements

Where Controllers Live

Servo controllers naturally operate in joint space. They read encoder values and drive joints toward commanded positions

Direct Hardware Interface

Motor commands, joint limits, and trajectory interpolation are all expressed in joint space

Task Space

The Space of End-Effector Pose

Describes where the robot's tool is in 3D space — position (x, y, z) and orientation. This is the space humans and vision systems think in

Where Vision & Planning Live

Vision outputs object pose in task space. Motion planning specifies goals in task space — "go to this position with this orientation"

Natural for Pick-and-Place

Pick-and-place tasks are most naturally stated as task-space goals: reach the object's detected pose

The Bridge Between Two Spaces



Degrees of Freedom (DOF)

The number of independent variables needed to fully describe the robot's configuration.

- Full 3D pose requires 6 DOF
- 6-DOF arms are the most common configuration
- More DOF → more flexibility, more complexity

Why 6 DOF?

In 3D space, a rigid body has 6 degrees of freedom: 3 translational (x, y, z) and 3 rotational (roll, pitch, yaw). A 6-DOF arm can theoretically achieve any pose within its workspace.

- ① 7-DOF arms add redundancy, enabling obstacle avoidance while reaching the same target.

Workspace



The Set of Reachable Points

Not every object position detected by vision is actually reachable by the robot. The workspace is the first hard constraint to check before executing any grasp.

- Dexterous workspace: positions reachable with all orientations
- Reachable workspace: positions reachable with at least one orientation
- Vision must validate reachability before commanding motion

Joint Limits

Physical Boundaries

Every joint has a maximum and minimum angle or travel. The robot physically cannot move beyond these limits

Valid IK Must Respect Limits

A mathematically valid IK solution that violates joint limits is not executable. Hardware safety systems will reject it

Theory vs. Reality

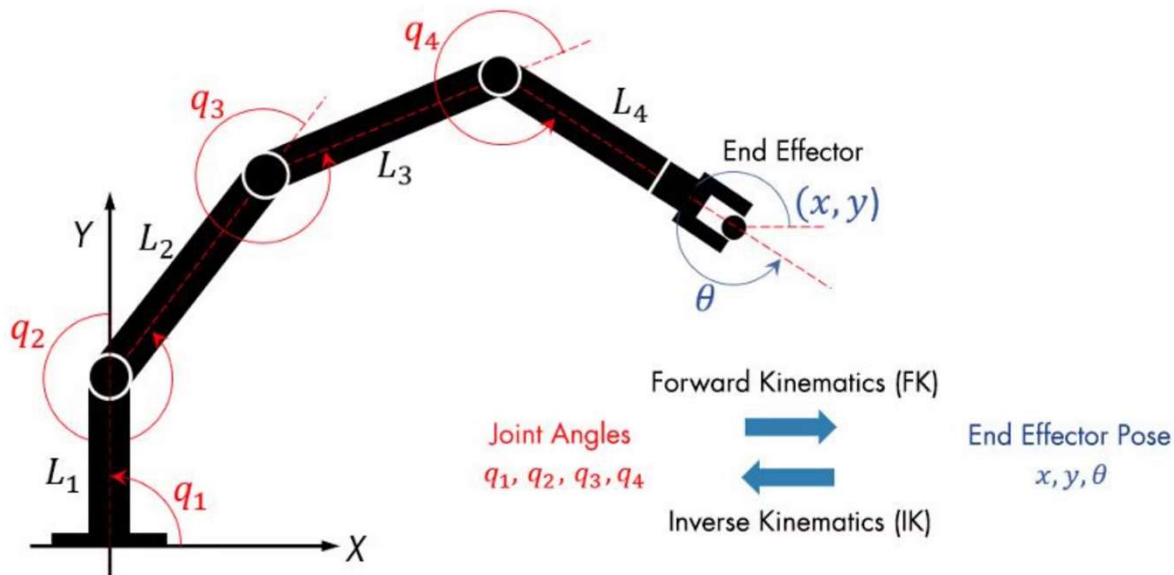
Joint limits are one of the biggest gaps between idealized robot math and real robot behavior in the lab

Part 3

Forward Kinematics

From joint angles to end-effector pose — the foundational forward mapping

What Is Forward Kinematics?



The Forward Problem

Given all joint angles (or displacements), compute the resulting **end-effector pose** in the base frame.

- ✓ FK is always solvable — there is exactly one end-effector pose for every joint configuration.

This makes FK the most reliable building block in arm kinematics. It has a unique, deterministic answer.

Practical Significance of FK



Know Where the Robot Is

Read encoder values → run FK → know the exact current pose of the tool in space



Monitoring & Visualization

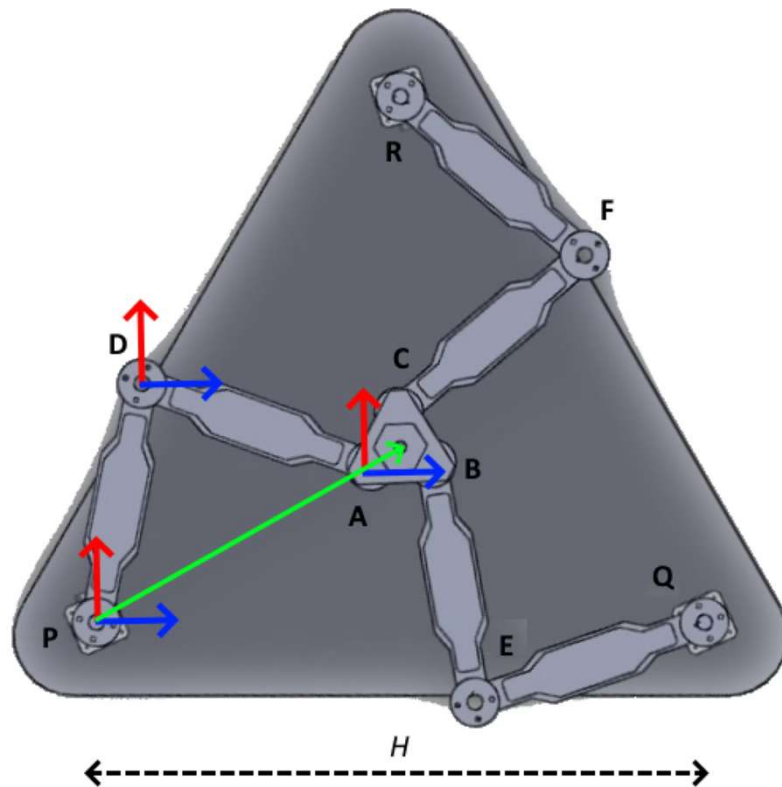
Drives real-time robot simulation, 3D display, and pose logging in development tools



Verify IK Solutions

Run FK on the IK output to confirm the computed joint angles actually produce the desired pose — a critical lab step

FK Intuition: 2R Planar Robot



The Simplest Case

A 2-link planar robot is the most intuitive FK example. It builds exactly the right mental model before tackling 6-DOF arms.

2R Planar FK — Closed Form

End-Effector Position

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

$$y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

Reading the Equations

Each term clearly shows the contribution of one link. l_1 and l_2 are link lengths. The equations reveal exactly how each joint angle influences the final TCP position — a direct, tangible connection between joints and task space.

FK Using DH Parameters

1

Four Parameters per Link

a (link length), α (link twist), d (link offset), θ (joint angle)

2

Classical & Widely Used

Denavit-Hartenberg notation is the traditional standard in textbooks and industrial robot documentation

3

Systematic for Serial Chains

Useful for organizing parameters of serial robot arms in a structured, consistent table

□ Week 7 will work through DH parameter tables in detail. This week we focus on the big picture.

FK as a Matrix Chain

The General Principle

Regardless of the parameterization used, FK is always a chain of matrix multiplications from base to tool.

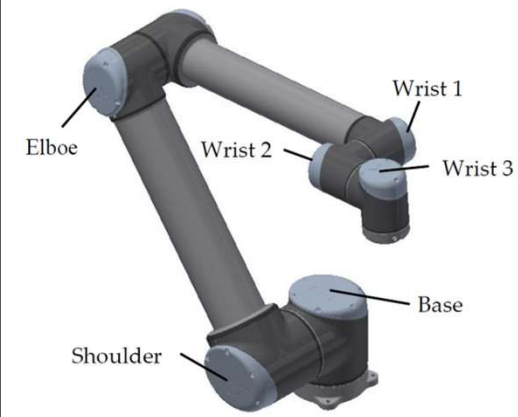
$$T_{0 \rightarrow n} = T_{01} T_{12} T_{23} \dots T_{(n-1)n}$$

Key Insight

DH notation is one way to fill in those matrices — but the underlying idea is universal. Any FK method is computing this chain product.

 This is why FK is easy to implement in code: it is just a loop of matrix multiplications.

FK for a 6-DOF Arm

	Joints		
	Name	Max. Joints Limits -/+ [°]	Max. Joints Velocity [°/s]
	Base	-363/363	131
	Shoulder	-363/363	131
	Elbow	-363/363	191
	Wrist 1	-363/363	191
	Wrist 2	-363/363	191
	Wrist 3	-363/363	191

Beyond the Simple 2R Case

With 6 joints, closed-form inspection becomes impractical. We rely on libraries and general matrix models.

- Use `fkine(q)` from Robotics Toolbox to compute FK
- Same matrix chain concept — just 6 steps instead of 2
- Week 7 will go deep on this topic

Part 4

Product of Exponentials

The modern geometric approach to kinematics — as emphasized in the RCV course

Why RCV Emphasizes PoE

Course Directive

The RCV material explicitly prioritizes **Product of Exponentials** as the Week 6 framework — lighter and more intuitive than full DH tables

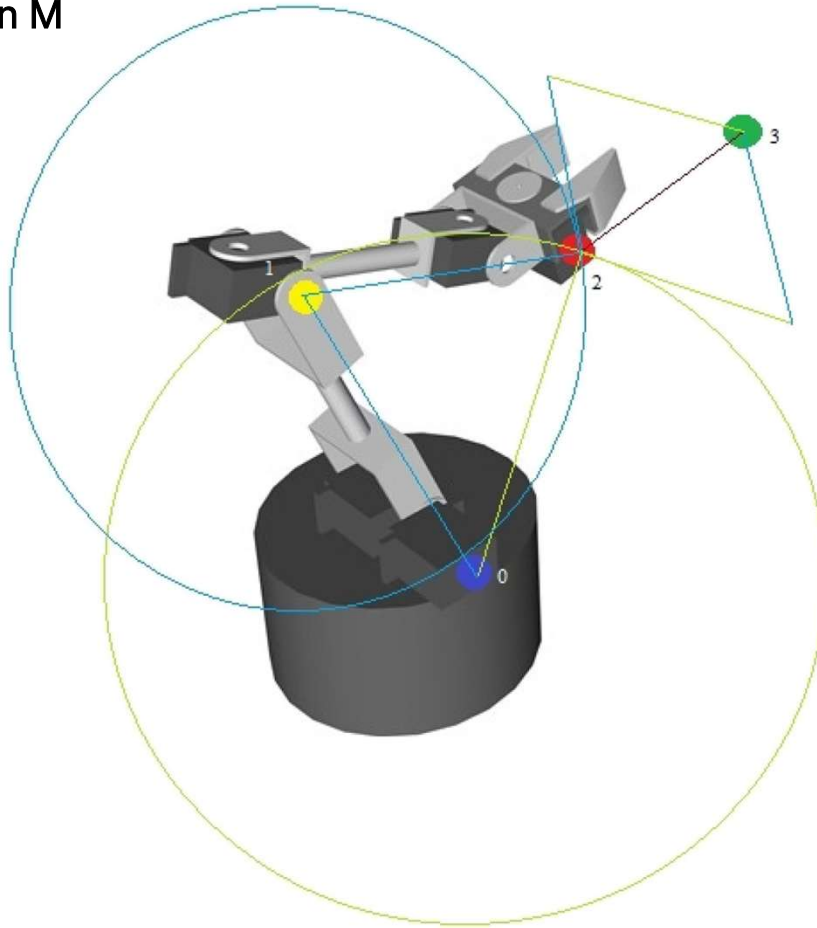
AI-Friendly Approach

PoE aligns with modern geometric thinking. Less bookkeeping, more insight — appropriate for students whose primary domain is AI, not mechanical engineering

Toolbox Compatible

PoE maps directly to simulation and toolbox implementations, making lab integration straightforward

Zero Configuration M



The Reference Posture

Before applying any joint motion, we define a **zero configuration** — a fixed reference posture for the robot.

- Denoted as the 4×4 matrix M
- Represents the end-effector pose when all joints are at zero
- All PoE computations are relative to this anchor

Choosing a convenient zero configuration simplifies all subsequent kinematics.

Screw Axis & Twist

The Language of PoE

Every joint is described by a **screw axis** — a unified way to represent both rotation and translation as a single geometric entity.

Twist Vector

A **twist** ξ combines angular and linear velocity into one 6-vector. Exponentiating a twist gives the rigid body transform produced by that joint's motion.

$$\xi = \begin{bmatrix} \omega \\ v \end{bmatrix} \in \mathbb{R}^6$$

Revolute

Rotates around the screw axis

Prismatic

Translates along the screw axis

PoE Formula

The Product of Exponentials

The end-effector pose is the product of exponential joint motions applied to the zero configuration **M**:

$$T(\theta) = e^{[S_1]\theta_1} \dots e^{[S_n]\theta_n} M$$

Reading the Formula

- Each $e^{[S_i]\theta_i}$ represents the contribution of one joint
- Multiplying them chains all joint motions together
- The result is applied to zero configuration **M**
- Intuition: each joint adds one "twist" of motion to the chain

Benefits of the PoE Approach



Modern Geometric Thinking

Aligns with how contemporary robotics textbooks (e.g., Modern Robotics) and libraries treat kinematics



No DH Table Required

Avoids the tedious frame-by-frame bookkeeping of DH parameters — students focus on geometric intuition instead



Toolbox & Simulation Ready

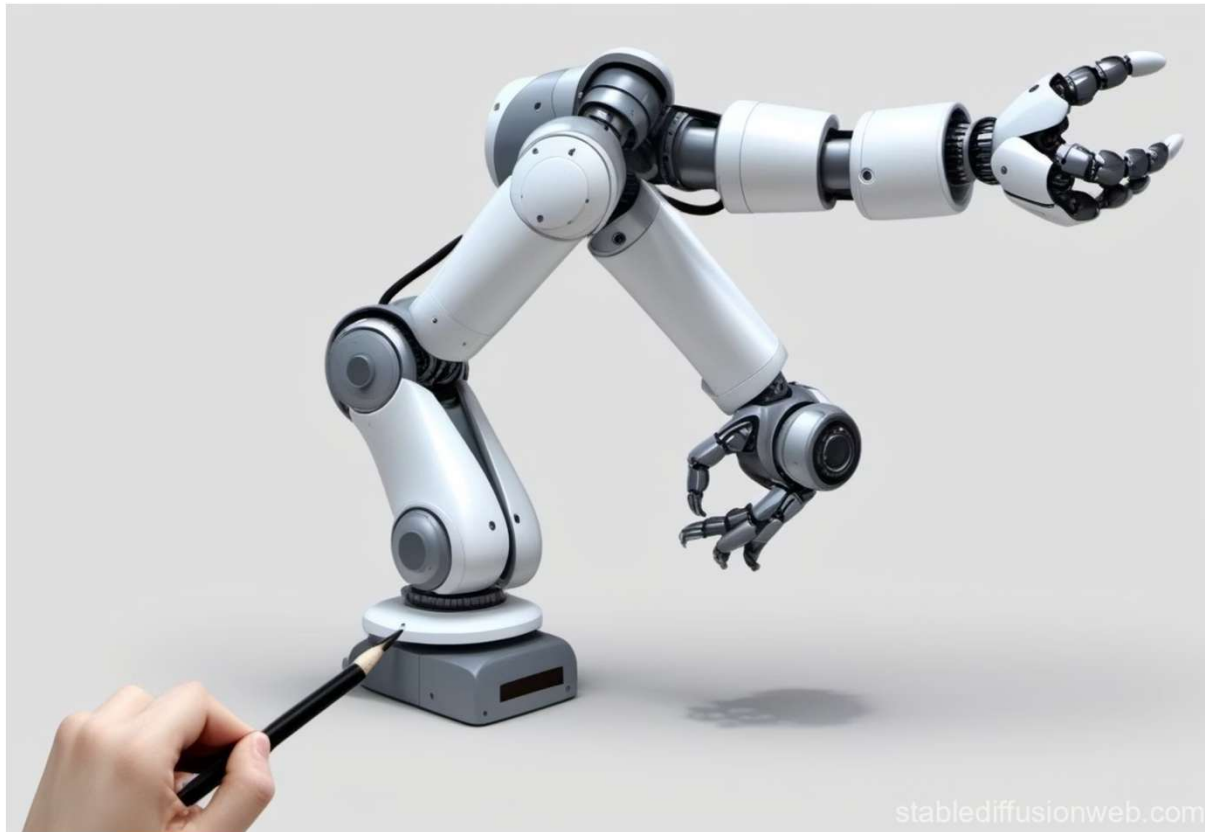
Maps directly to Python Robotics Toolbox and other modern simulation environments used in lab

Part 5

Inverse Kinematics

Finding the joint configuration that achieves a desired end-effector pose

What Is Inverse Kinematics?



The Inverse Problem

Given a **desired end-effector pose**, find the joint configuration $\mathbf{q}$ that achieves it. IK is the most direct link between vision output and robot action.

$$T_{target} \xrightarrow{IK} \mathbf{q} = [q_1, q_2, \dots, q_n]$$

This is the step that translates "the object is here" into "move joint 1 to 45°, joint 2 to -30°..."

Why IK Is Harder Than FK

Multiple Solutions

One target pose may be reachable via several different joint configurations — the robot must choose wisely

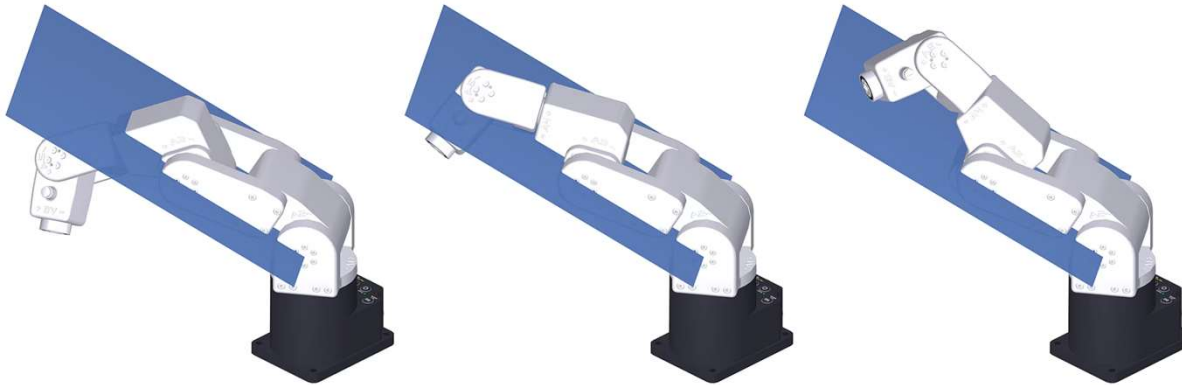
No Solution

The target may be outside the workspace, or the required orientation may be physically impossible given the robot's structure

Constrained Solutions

Even valid mathematical solutions may violate joint limits, collision constraints, or physical feasibility in the real lab

Multiple IK Solutions



Example: Elbow-Up vs. Elbow-Down

For a typical 6-DOF arm, a single target pose can have up to 16 IK solutions. The robot must select the most appropriate one.

- Choose the solution closest to the current joint configuration
- Avoid configurations that pass through singularities
- Respect joint limits and workspace boundaries

When IK Has No Solution

Target Outside Workspace

The object is too far or too close — the robot arm simply cannot physically reach it regardless of joint configuration

Orientation Infeasible

Vision requests an approach angle that the robot's wrist geometry cannot physically achieve

Joint Limits Block the Path

Even if a mathematical solution exists, joint limits make it physically unreachable — the target is no longer feasible

 Always verify IK feasibility before commanding robot motion. A failed IK must trigger a graceful fallback, not a crash.

Analytical vs. Numerical IK

Analytical IK

Fast and exact — derives closed-form equations for specific robot geometries. Works well for standard arm configurations with known structure.

- Computationally cheap
- Limited to specific geometries
- Difficult to derive for arbitrary arms

Numerical IK

General purpose — uses iterative optimization to find solutions for any robot. Preferred for lab work and software libraries.

- Works for any robot configuration
- Slightly slower but very practical
- Basis of `ikine()` in Robotics Toolbox

📅 Week 8 will cover both approaches in depth with lab exercises.

IK in the RCV Course

Pragmatic Approach for AI Students

The RCV course recommends using `ikine()` from the Robotics Toolbox so students can focus on **application logic** rather than hand-solving IK equations.

Core Goal

Understand the full pipeline: **perception** → **pose** → **IK** → **motion**

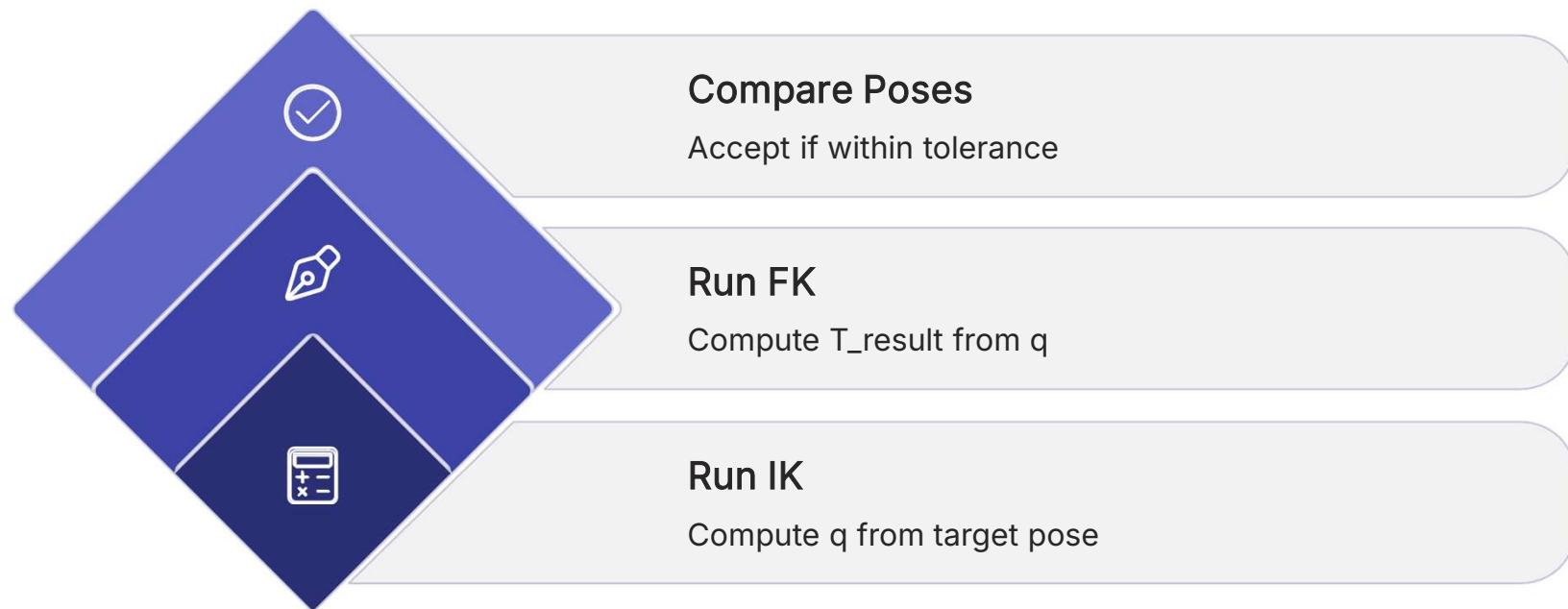
Not: solve IK by hand.

→ Call `ikine(T_target)` to get joint angles

→ Verify with `fkine(q)` to confirm the result

→ Focus on the perception-to-motion pipeline logic

Verifying an IK Solution



Never blindly trust an IK result. Always verify by running FK on the returned joint angles. If $FK(q)$ is close to the target pose within tolerance, the solution is valid. This check is mandatory in every lab implementation.

Part 6

Jacobian & Control

The differential relationship between joint motion and end-effector motion

What Is the Jacobian?

The Differential Model

The Jacobian matrix $J(q)$ relates **joint velocities** to **end-effector velocity**. It is the local linear approximation of the FK function around the current configuration.

$$\dot{x} = J(q)\dot{q}$$

Why It Matters

- Core tool for velocity-level control
- Essential for visual servoing
- Reveals singularities and manipulability
- Basis for Jacobian-based IK (numerical methods)

Jacobian Intuition

Each Column = One Joint's Contribution

Column i of J shows how joint i alone would move the TCP — direction and magnitude of influence

Translation vs. Rotation

Some joints contribute mainly to TCP translation; others dominate TCP rotation — the Jacobian shows this quantitatively

Configuration-Dependent

$J(q)$ changes as the robot moves. A joint near a singularity has very different influence than the same joint in a well-conditioned pose

Resolved-Rate Control

From Desired TCP Velocity to Joint Velocities

If we know the desired TCP velocity $\dot{\mathbf{x}}$, we can solve for the required joint velocities using the Jacobian inverse (or pseudoinverse):

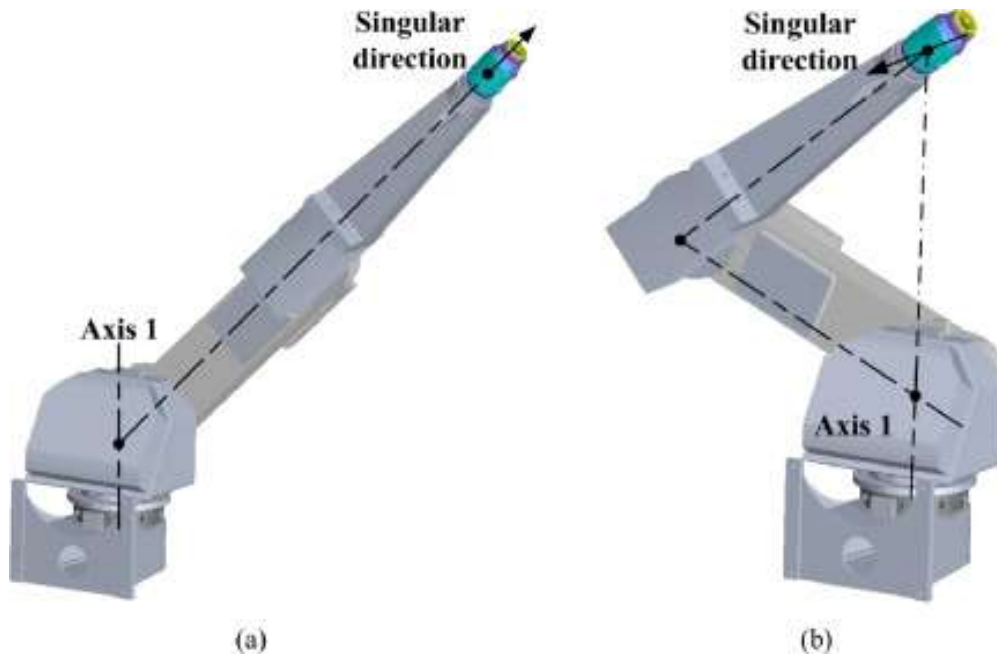
$$\dot{\mathbf{q}} = \mathbf{J}^+(\mathbf{q}) \dot{\mathbf{x}}$$

Why This Matters for Vision

In visual servoing, vision continuously provides an error signal in task space. Resolved-rate control converts that error into joint commands — a natural, elegant integration of vision and control.

- ✓ This is the conceptual bridge from perception to real-time robot motion.

Singularity



When the Robot Loses Mobility

At certain configurations, the robot loses the ability to move freely in some directions. The Jacobian becomes rank-deficient (or near rank-deficient).

- Occurs at workspace boundaries or fully extended configurations
- Jacobian pseudoinverse amplifies noise enormously near singularities
- Source of instability in IK and velocity control

Singularity in the Lab

Large Joint Motion, Small TCP Motion

The robot commands large joint velocity but the tool barely moves — energy is wasted, control is inefficient

Oscillation or Non-Convergence

The robot shakes, vibrates, or fails to reach the target smoothly — a clear sign of singularity proximity

Trajectory Discontinuities

Path becomes unsmooth or erratic. The robot may jump unexpectedly or generate unsafe velocity commands

 Always plan trajectories that avoid known singularity regions. Use damped least squares (DLS) pseudoinverse near singularities.

Position Control vs. Velocity Control

Position Control

Commands a specific target joint angle. The servo drives the joint to that position.

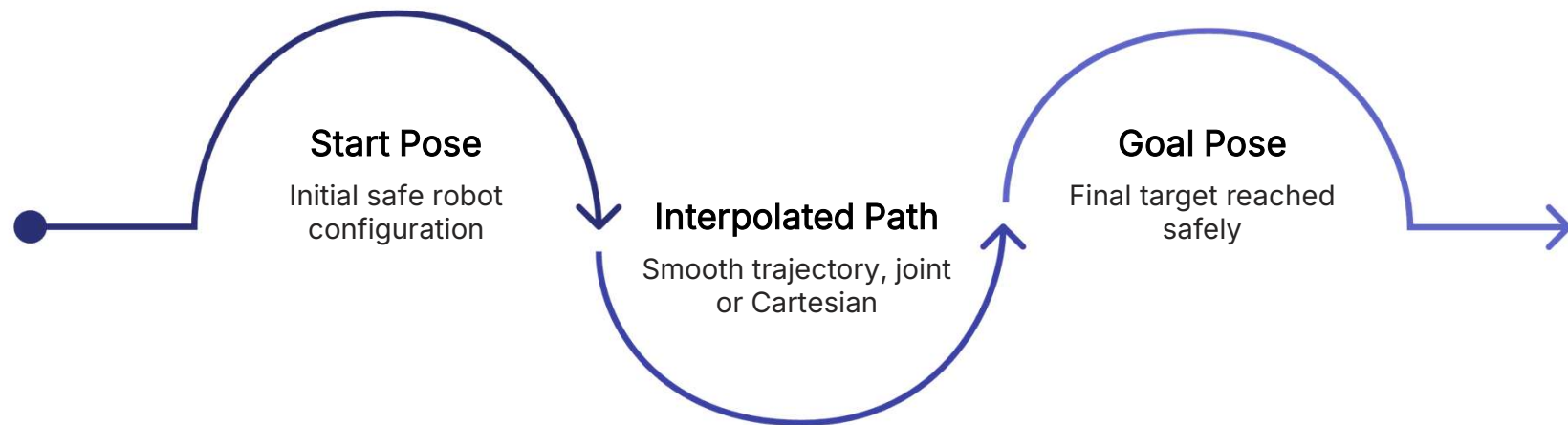
- Ideal for point-to-point motion
- Simple and stable
- Most industrial robots default to this

Velocity Control

Commands joint velocity. The controller continuously adjusts based on feedback.

- Ideal for continuous target tracking
- Natural for visual servoing
- Requires careful tuning to avoid instability

Trajectory, Not Jumps



Never command the robot to jump directly between two poses. Trajectory interpolation (joint-space or Cartesian) ensures smooth, safe, and stable motion. This is critical for both physical safety and IK numerical convergence during continuous tracking tasks.

Part 7

Mapping Vision to Robot

Closing the loop — from camera coordinates to executable robot commands

Camera Frame → Robot Base Frame



The Mandatory Transformation

Vision outputs object pose in the **camera frame**. The robot executes motion in its **base frame**. A correctly calibrated transform between them is not optional — it is the prerequisite for any successful grasp.

Without this transform, even perfect vision output will cause the robot to reach the wrong location.

Hand-Eye Calibration

Establishing the Camera-Robot Relationship

Hand-eye calibration determines the rigid-body transform between the camera frame and the robot frame — the most critical calibration step in vision-guided robotics.

Eye-in-Hand

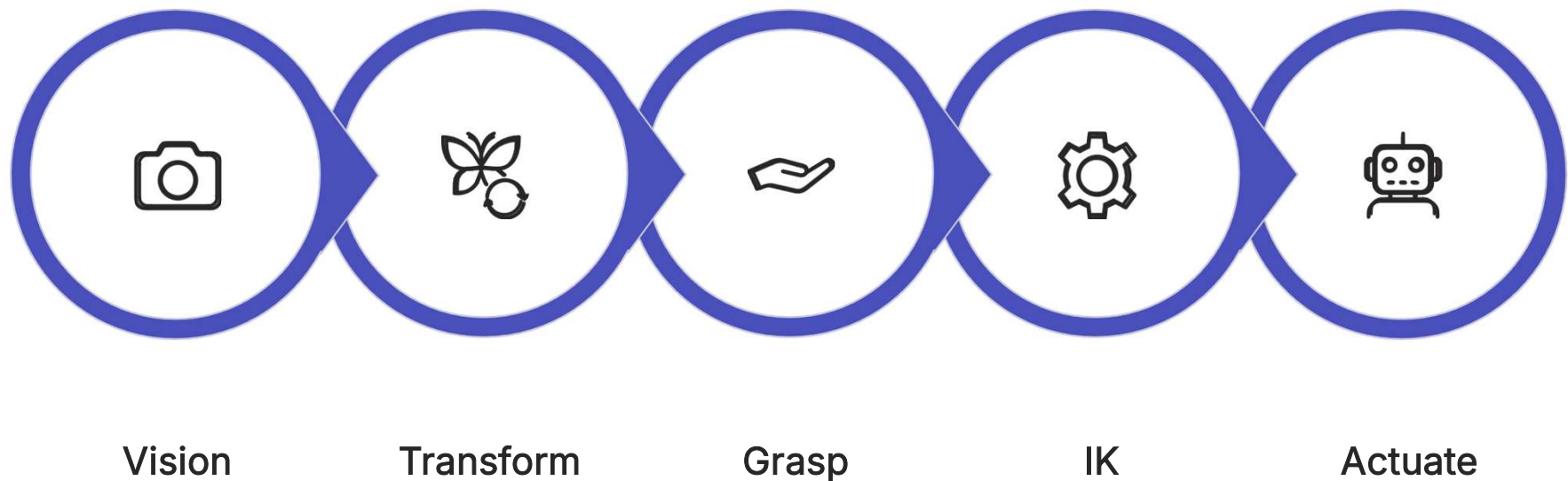
Camera mounted on the end-effector — moves with the robot. Good for close-up, dynamic grasping

Eye-to-Hand

Camera fixed in the workspace — observes the robot from outside. Good for wide-area scene understanding

📅 Week 9 will cover hand-eye calibration procedures and implementation in detail.

End-to-End Pipeline



This is the complete vision-to-action pipeline. Each stage depends on the one before it. Weeks 7, 8, and 9 will deepen each component — but understanding this pipeline as a whole is the most important outcome of Week 6.

Week 6 — Summary

Framework Established

Week 6 sets the conceptual architecture for all robot manipulation tasks

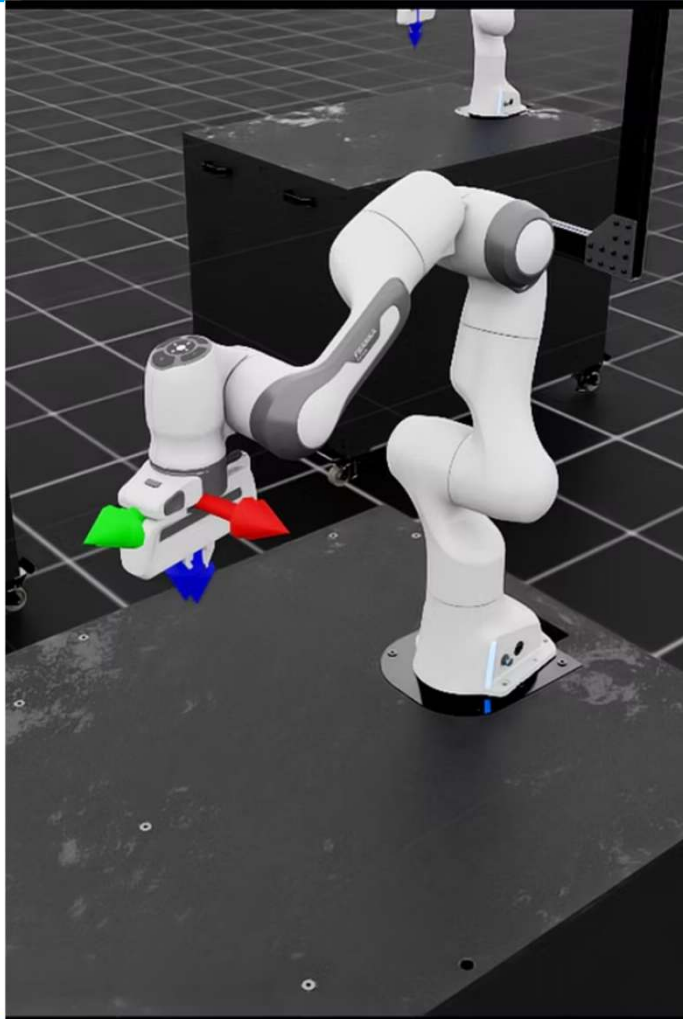
Depth Coming Later

FK → Week 7 · IK → Week 8 ·
Camera Mapping → Week 9

The Key Flow

Vision → Pose → Frame Transform → IK → Control → Robot Action





Lab Week 6: Kinematics & Mapping

Bridge your Week 5 vision output (6D pose in camera frame) to a robot target — by mastering $SE(3)$ transforms, frame changes, and Jacobian intuition. This lab lays the essential groundwork for FK (Week 7) and IK (Week 8).

🕒 90–100 minutes

[PYTHON](#) · [SPATIALMATH](#) · [ROBOTICSTOOLBOX](#)

What You'll Accomplish Today

This lab is structured as five focused modules, each building on the last. By the end, you'll have a complete pipeline from a camera-detected object pose to a robot-ready target transform.



Lab 6.1 — Play with SE(3)

Build intuition for rotations, translations, and composed transforms using `spatialmath`.



Lab 6.2 — Frame Change

Convert object poses from camera frame to robot base frame using a known calibration transform.



Lab 6.3 — PoE & FK Demo

Run Forward Kinematics via `roboticstoolbox` and observe how joint configs drive TCP pose.



Lab 6.4 — Jacobian Preview

Compute joint velocities from a desired TCP velocity using the pseudo-inverse Jacobian.



Lab 6.5 — Full Pipeline

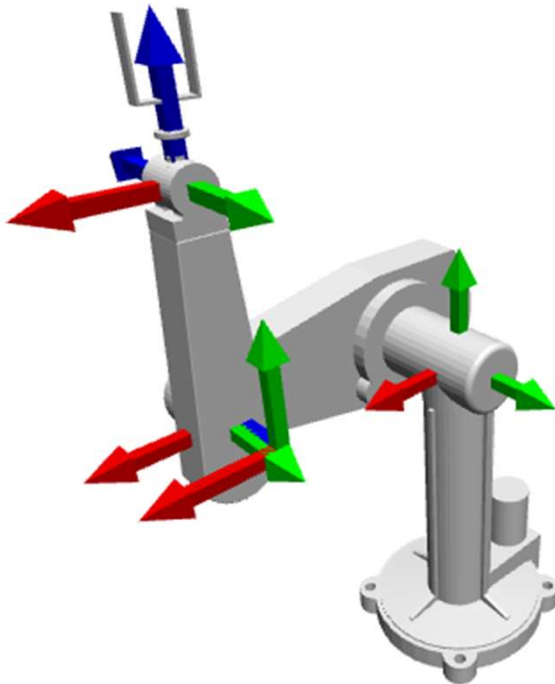
Assemble the complete Vision → Pose → Robot Target chain, ready for IK in Week 8.

LAB 6.1

⌚ ~20 minutes

Playing with SE(3) via spatialmath

SE(3) — the Special Euclidean group in 3D — represents any rigid-body transform: rotation + translation. You'll construct transforms, apply them to points, and verify inverse correctness.



🔄 Rotation

Create rotations around X, Y, Z axes using `SE3.Rx()`, `SE3.Ry()`, `SE3.Rz()` with angles in radians.

➡ Translation

Build translations using `SE3.Tx()`, `SE3.Ty()`, `SE3.Tz()` and compose them with the `*` operator.

✅ Inverse Check

Verify that `T.inv() * T * p ≈ p`. This sanity check confirms your transform is valid and reversible.

SE(3) in Action: Rotation + Translation + Inverse

Run this starter code, then create **at least 3 more transform examples** on your own. Try combining Rx, Ry, Tz in different orders — notice how order matters!

```
from spatialmath import SE3
import numpy as np

# Rotation 45° around Z axis
Rz = SE3.Rz(np.deg2rad(45))

# Translation: 0.2m along X, 0.1m along Y
T_trans = SE3.Tx(0.2) * SE3.Ty(0.1)

# Compose: translate THEN rotate
T = T_trans * Rz

p = np.array([0.1, 0.0, 0.0]) # Point 10cm on X-axis
p_new = T * p

print("T =\n", T.A)
print("p_new =", p_new)

# Inverse check: T_inv * (T * p) should equal p
p_recovered = T.inv() * p_new
print("Recovered p =", p_recovered) # Should match original p
```

  **Key insight:** Transform composition is NOT commutative. $T_trans * Rz \neq Rz * T_trans$. Try swapping the order and see what changes!

Frame Change: Camera → Robot Base

The Setup

After hand-eye calibration, we know $T_{\text{base_cam}}$ — the fixed transform from camera frame to robot base frame. Given an object's pose in camera frame $T_{\text{cam_obj}}$, the object in base frame is:

$$T_{\text{base_obj}} = T_{\text{base_cam}} \times T_{\text{cam_obj}}$$

This is the chain rule of transforms — the core operation you'll use in every robotics pipeline.

Why This Matters

Your Week 5 output gives you $T_{\text{cam_obj}}$ — the object's 6D pose relative to the camera. But the robot only understands positions in its own **base frame**. This conversion is the essential bridge between vision and manipulation.

- Camera frame: where the object *appears*
- Base frame: where the robot needs to *go*
- $T_{\text{base_cam}}$: the calibrated bridge between them



 **Deliverable:** Implement `to_base_frame( $T_{\text{base\_cam}}$ ,  $T_{\text{cam\_obj}}$ )` as a reusable function.

LAB 6.2 — CODE

Frame Conversion: Full Example

```
from spatialmath import SE3
import numpy as np

# Camera is 0.3m above base, rotated 90° around Y
# (This comes from your hand-eye calibration result)
T_base_cam = SE3.Tz(0.3) * SE3.Ry(np.deg2rad(90))

# Object detected in camera frame (placeholder from Week 5)
T_cam_obj = SE3.Tx(0.15) * SE3.Tz(0.05) * SE3.Rz(np.deg2rad(30))

# Transform to base frame – the key operation
T_base_obj = T_base_cam * T_cam_obj

print("Object in base frame:\n", T_base_obj.A)

# --- Your deliverable ---
def to_base_frame(T_base_cam, T_cam_obj):
    """Convert object pose from camera frame to robot base frame."""
    return T_base_cam * T_cam_obj

# Test it
result = to_base_frame(T_base_cam, T_cam_obj)
print("Via function:\n", result.A)
```

⚠ **Common mistake:** Don't flip the multiplication order! $T_{cam_obj} * T_{base_cam}$ gives a completely wrong result. Think of it as: "start from base, go to cam, go to obj."

LAB 6.3

⌚ ~20 minutes

PoE & Forward Kinematics via Toolbox

Instead of implementing Product of Exponentials from scratch, we use `roboticstoolbox` as a practical demonstration. The math details come in Week 7 — today, focus on *seeing* FK work: give it joint angles, get back a TCP pose.

Choose a Robot Model

Use `rtb.models.ETS2.planar2()` — a simple 2-DOF planar robot. If your lab has a Yahboom model, try that instead for real-world relevance.

Vary Joint Configs

Try $q = [\theta, \theta], [\pi/2, \theta], [\theta, \pi/2]$, and combinations. Watch how `fkine(q)` changes the TCP transform.

Read the Output

The 4×4 matrix from `T_fk.A` is an SE(3) transform. The top-left 3×3 is rotation; the right column (xyz) is position — now familiar from Lab 6.1!

LAB 6.3 — CODE

FK with roboticstoolbox

```
import roboticstoolbox as rtb
import numpy as np

# Load a simple 2-DOF planar robot
robot = rtb.models.ETS2.planar2()

# Try different joint configurations
configs = {
    "home": np.array([0.0, 0.0]),
    "elbow": np.array([0.5, 0.5]),
    "reach": np.array([0.2, -0.3]),
}

for name, q in configs.items():
    T_fk = robot.fkine(q)
    print(f"\nConfig '{name}': q = {q}")
    print("TCP pose T_fk =\n", T_fk.A)
```

What to observe

- How does changing $q[0]$ alone affect TCP position?
- When is the arm fully extended?
- Can you find a config where the TCP is back near the base?

Looking ahead

Week 7 will derive *why* this works using the Product of Exponentials formula. For now, treat `fkine()` as a black box that maps joint space $\rightarrow$ task space.

LAB 6.4

 ~20 minutes

Jacobian & Resolved-Rate Control (Preview)

The Jacobian $\mathbf{J}(\mathbf{q})$ maps joint velocities to TCP velocities: $\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$. Inverting this lets us ask: "what joint speeds produce the TCP motion I want?" This is the foundation of velocity-level robot control.



Compute $\mathbf{J}(\mathbf{q})$

Use `robot.jacob0(q)` to get the Jacobian at a specific configuration. For `planar2`, this is a 2×2 matrix — each column shows how one joint contributes to TCP motion.



Pseudo-Inverse

Use `np.linalg.pinv(J)` to compute $\mathbf{J}^+$. Multiply by desired TCP velocity to get joint velocities: $d\mathbf{q} = \mathbf{J}^+ \cdot \mathbf{v}$. Works even when $\mathbf{J}$ is not square.



Build Intuition

You don't need to prove where $\mathbf{J}$ comes from today. Focus on the result: inspect $d\mathbf{q}$ and ask yourself which joint moves more when TCP moves along X vs. Y.

```

import roboticstoolbox as rtb
import numpy as np

robot = rtb.models.ETS2.planar2()
q = np.array([0.5, 0.3])

J = robot.jacob0(q) # 2x2 Jacobian for planar2
v = np.array([0.01, 0.0]) # Desired TCP: 1 cm/s along X
dq = np.linalg.pinv(J) @ v # Required joint velocities

print("J =\n", J)
print("dq =", dq) # rad/s for each joint

```

Full Pipeline: Vision → Pose → Robot Target

This final module ties everything together. Simulate the Week 5 object detection output, convert to base frame, and produce `T_target` — the exact input your IK solver will consume in Week 8.



This pipeline is your capstone for Weeks 5–9. Each week adds one more link in the chain from raw camera pixels to robot motion.

```
from spatialmath import SE3
import numpy as np

# Step 1: Known calibration (hand-eye result)
T_base_cam = SE3.Tz(0.3) * SE3.Ry(np.deg2rad(90))

# Step 2: Object pose from Week 5 vision (placeholder)
T_cam_obj = SE3.Tx(0.15) * SE3.Tz(0.05) * SE3.Rz(np.deg2rad(30))

# Step 3: Convert to base frame
T_base_obj = T_base_cam * T_cam_obj

# Step 4: Define grasp target (Week 8 will add tool offset)
T_target = T_base_obj

print("T_target (input for IK in Week 8):\n", T_target.A)
```

🔗 **Week 8 extension:** You'll add a grasp offset transform between `T_base_obj` and `T_target` to account for gripper geometry. Keep this file — you'll reuse it directly!

Lab Summary & Key Takeaways



SE(3) is your language

Every pose in robotics is a 4×4 homogeneous matrix. Compose with `*`, invert with `.inv()`.



Frame change is just multiplication

`T_base_obj = T_base_cam * T_cam_obj`. The chain rule of transforms is the single most-used operation in robotics.



FK = joint config → TCP pose

`fkine(q)` gives you the end-effector SE(3) transform. Week 7 will explain the math; today you built the intuition.



Jacobian connects velocities

The pseudo-inverse Jacobian converts desired TCP velocity to joint velocities — the core of resolved-rate and IK solvers.

 **Coming up:** Week 7 derives FK and PoE from first principles. Week 8 solves IK and uses your `T_target` directly. This lab is the connective tissue — make sure your `to_base_frame()` function and `T_target` code are saved and tested!